LABORATORY

LAB 11

Motion Data Monitoring with MPU6050

Measuring Acceleration and Angular Velocity with MPU6050 via EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction . . . 2
1.1 Lab Overview . . . 2
1.2 Objectives . . . 2
2 Background . . . 2
2.1 Accelerometer and Gyroscope . . . 2
2.2 I2C Communication . . . 3
3 Hardware Setup . . . 3
3.1 Required Hardware . . . 3
3.2 I2C Signal Mapping . . . 3
3.3 Connecting the MPU6050 . . . 3
4 EduFramework API . . . 4
4.1 MPU_Data_t Structure . . . 4
4.2 MPU_Begin() . . . 5
4.3 MPU_ReadData() . . . 5
5 Lab Exercise . . . 6
5.1 Requirements . . . 6
5.2 Program . . . 7
5.3 Program Explanation . . . 8
5.4 Verification . . . 8
6 Extension . . . 8
6.1 Selective Reading and Status Checking . . . 8
6.2 Calibration with MPU_Calibrate() . . . 9
6.3 I2C Address and the AD0 Pin . . . 9
6.4 Customizing Measurement Ranges . . . 10
6.5 Context-Based Advanced API . . . 10
6.6 Extension Exercise . . . 11
7 References . . . 11

1 Introduction

1.1 Lab Overview

The MPU6050 is a six-axis motion sensor integrating a three-axis accelerometer and a three-axis gyroscope. It measures specific force and angular velocity along the X, Y, and Z axes, providing information about the motion of the object to which the sensor is attached [3] .

This lab uses the EduFramework Device API instead of directly accessing the MPU6050 registers. The program initializes the sensor, reads acceleration, angular velocity, and internal sensor temperature, and displays the measurements on the Serial Monitor. The Extension section introduces selective reading, calibration, I2C address selection, and measurement-range configuration [6] .

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the roles of the accelerometer and gyroscope in a six-axis motion sensor.
2. Explain the basic elements of I2C communication, including `SDA` , `SCL` , and device addressing.
3. Use the `MPU_Data_t` structure to access acceleration, angular velocity, and sensor-temperature data provided by EduFramework.
4. Use `MPU_Begin()` and `MPU_ReadData()` to develop an MPU6050 motion-data monitoring application on the Serial Monitor.
5. Identify the advanced configuration capabilities of the MPU6050 Device API, including calibration, I2C address selection, and measurement-range selection.

2 Background

2.1 Accelerometer and Gyroscope

The MPU6050 is an inertial measurement unit (IMU) containing a three-axis accelerometer and a three-axis gyroscope. The accelerometer measures **specific force**, whereas the gyroscope measures angular velocity about each axis. These measurements provide complementary information for observing sensor motion [1] [3] .

In EduFramework, acceleration is expressed in `g` (standard gravitational acceleration), while angular velocity is expressed in `dps` (degrees per second). When the sensor is stationary, the accelerometer still responds to gravity: an axis nearly aligned with the vertical direction may have a magnitude of approximately `1 g` . In contrast, angular-velocity readings are generally close to `0 dps` , although zero-rate offset and noise may remain [1] [6] .

Note

A gyroscope measures angular velocity rather than rotation angle directly. Estimating orientation angles such as roll, pitch, or yaw requires additional data processing and is beyond the scope of this lab.

2.2 I2C Communication

I2C is a synchronous serial communication bus with two signal lines: `SCL` (Serial Clock) carries the clock signal, and `SDA` (Serial Data) carries data. Multiple devices can share these lines, with the controller distinguishing individual devices by their I2C addresses [2] .

In this lab, the S32K144 communicates with the MPU6050 over I2C. The `AD0` pin determines the least significant bit of the 7-bit I2C address: a logic-low level selects `0x68` , whereas a logic-high level selects `0x69` [3] .

The application data flow is:

`MPU6050` → `I2C` → `Wire / LPI2C` → `MPU6050 Device API` → `Application`

3 Hardware Setup

3.1 Required Hardware

The lab uses the MaaZEDU Development Board and an MPU6050 module. MaaZEDU is a development board based on the NXP S32K144 microcontroller [5] [4] . The sensor module is supplied with `3.3V` , shares `GND` with the board, and exchanges data through the I2C interface [3] .

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
MPU6050 module	Sensor module integrating a three-axis accelerometer and a three-axis gyroscope.
Jumper wires	Connect power and I2C signals between the MPU6050 and MaaZEDU.
USB cable	Connect the board to the computer for power, program upload, and Serial Monitor access.

Table 11.3.1. Hardware required for the lab exercise

3.2 I2C Signal Mapping

In the EduFramework configuration used for this lab, the I2C0 bus is mapped to `I2C0_SCL` (`PTA3`) and `I2C0_SDA` (`PTA2`). The Wire interface and MPU6050 driver use these pins to exchange data [6] .

Component	Logical Pin	MCU Pin	Function
MPU6050 `SCL`	`I2C0_SCL`	`PTA3`	I2C clock
MPU6050 `SDA`	`I2C0_SDA`	`PTA2`	I2C data

Table 11.3.2. MPU6050 signal mapping used in the lab exercise

3.3 Connecting the MPU6050

Connect the module's `VCC` pin to `3.3V` , `GND` to `GND` , `SCL` to `I2C0_SCL` , and `SDA` to `I2C0_SDA` . To use the default I2C address `0x68` , the `AD0` pin must be at a logic-low level. If the module does not include a pull-down circuit, connect `AD0` to `GND` ; verify the pinout of the actual module before applying power [3] [6] .

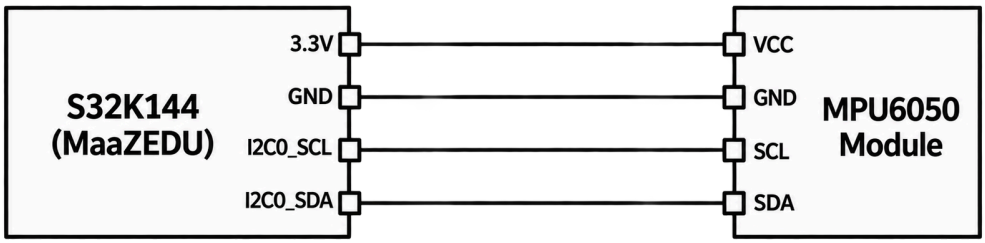


Figure 11.3.1. Four power and I2C connections between the MPU6050 and MaaZEDU

Note

The diagram shows the four main connections and does not depict the `AD0` pin. Before running the program, ensure that `AD0` is low to select address `0x68`. The `INT` pin is not used because the program polls measurements periodically using `MPU_ReadData()`.

4 EduFramework API

EduFramework provides a Beginner API for operating the MPU6050 with its default configuration and an Advanced API for managing configuration through a device context. The main exercise uses only `MPU_Data_t`, `MPU_Begin()`, and `MPU_ReadData()`. Additional APIs are introduced in the Extension section [6].

4.1 MPU_Data_t Structure

`MPU_Data_t` is a structure that stores measurements along the three axes and the internal sensor temperature. When `MPU_ReadData()` returns `true`, the application accesses individual structure members using the dot (`.`) operator [6].

```
typedef struct
{
    float accelX;
    float accelY;
    float accelZ;
    float gyroX;
    float gyroY;
    float gyroZ;
    float temperature;
} MPU_Data_t;
```

Listing 11.4.1. `MPU_Data_t` data structure

Field	Quantity	Unit
`accelX`	Specific force along the X-axis	`g`
`accelY`	Specific force along the Y-axis	`g`
`accelZ`	Specific force along the Z-axis	`g`
`gyroX`	Angular velocity about the X-axis	`dps`
`gyroY`	Angular velocity about the Y-axis	`dps`
`gyroZ`	Angular velocity about the Z-axis	`dps`
`temperature`	Internal MPU6050 sensor temperature	`°C`

Table 11.4.1. Fields of the `MPU_Data_t` structure

Note

The `temperature` field reports the temperature measured inside the MPU6050, not a direct measurement of the surrounding air temperature [3] .

4.2 MPU_Begin()

`MPU_Begin()` initializes the MPU6050 using the default EduFramework configuration. According to the project source code, the API initializes the Wire layer, uses address `0x68` , wakes the sensor, and selects an accelerometer range of `±2 g` and a gyroscope range of `±250 dps` [6] .

`MPU_Begin`

Initialize the MPU6050 using the default EduFramework configuration.

Syntax

```
MPU_Begin();
```

Return value

`true` if initialization and communication verification succeed; `false` if the sensor cannot be reached or configured.

Example:

```
if (true == MPU_Begin())
{
    /* MPU6050 is ready. */
}
```

4.3 MPU_ReadData()

`MPU_ReadData()` reads one measurement frame from the MPU6050 and updates the acceleration, angular-velocity, and temperature fields of `MPU_Data_t` . The function reports success only when the sensor has been initialized and the read operation completes without error [6] .

MPU_ReadData

Read a measurement frame and update the supplied data structure.

Syntax

```
MPU_ReadData(data);
```

Parameters

Parameter	Type / accepted value	Description
data	Address of the data storage location	Address of an `MPU_Data_t` variable that receives the measurements, such as `&data`.

Return value

`true` if the read succeeds; `false` if the sensor is not initialized, the argument is invalid, or I2C communication fails.
Example:

```
MPU_Data_t data;

if (true == MPU_ReadData(&data))
{
    Serial1_printFloat(data.accelX);
}
```

5 Lab Exercise

5.1 Requirements

Write a program that reads three-axis acceleration, three-axis angular velocity, and the internal MPU6050 temperature, then displays the results on the Serial Monitor. Initialize the sensor with `MPU_Begin()`; in the main loop, call `MPU_ReadData()` and wait `1000 ms` before the next iteration.
The Serial Monitor operates at `9600 baud`. If initialization fails, the program prints an error and does not continue reading. If an individual read fails, the program reports the error and retries during the next iteration.

5.2 Program

In `src/main.c`, use the program verified on the hardware:

```
#include "Arduino.h"
#include "MPU6050.h"

int main(void)
{
    MPU_Data_t data;
    setup();
    Serial1_begin(9600U);
    Serial1_println("MPU6050 Sensor Monitor");
    Serial1_println("----------------------");
    if (false == MPU_Begin())
    {
        Serial1_println("MPU6050 initialization failed.");
        while (1)
        {
            delay(1000U);
        }
    }
    Serial1_println("MPU6050 initialized.");
    while (1)
    {
        if (true == MPU_ReadData(&data))
        {
            Serial1_print("Accel X: ");
            Serial1_printFloat(data.accelX);
            Serial1_print(" g | Y: ");
            Serial1_printFloat(data.accelY);
            Serial1_print(" g | Z: ");
            Serial1_printFloat(data.accelZ);
            Serial1_println(" g");
            Serial1_print("Gyro X: ");
            Serial1_printFloat(data.gyroX);
            Serial1_print(" dps | Y: ");
            Serial1_printFloat(data.gyroY);
            Serial1_print(" dps | Z: ");
            Serial1_printFloat(data.gyroZ);
            Serial1_println(" dps");
            Serial1_print("Temperature: ");
            Serial1_printFloat(data.temperature);
            Serial1_println(" C");
            Serial1_println("----------------------");
        }
        else
        {
            Serial1_println("MPU6050 read failed.");
        }
        delay(1000U);
    }
}
```

Listing 11.5.1. MPU6050 motion-data monitoring program

5.3 Program Explanation

`setup()` initializes the underlying EduFramework components, and `Serial1_begin(9600U)` configures the Serial Monitor at `9600 baud`. `MPU_Begin()` then initializes the MPU6050. If the function returns `false`, the program reports the error and does not continue reading the sensor [6].

Each iteration calls `MPU_ReadData(&data)` to store the measurements in the `data` structure. When the function returns `true`, the program prints `accelX`, `accelY`, `accelZ`, `gyroX`, `gyroY`, `gyroZ`, and `temperature` using the Serial APIs. If the read fails, the program prints `MPU6050 read failed.` and retries on the next iteration [6].

The application flow can be summarized as:

`Motion` → `MPU6050` → `I2C` → `MPU_ReadData()` → `Serial1` → `Serial Monitor`

`delay(1000U)` introduces a one-second pause after each processing cycle. The actual update period also includes the time required to read the sensor and transmit the data over Serial.

5.4 Verification

Build and upload the program to the MaaZEDU board, then open the Serial Monitor at `9600 baud`.

First, place the module at rest on a stable surface. When one sensor axis is nearly aligned with gravity, the acceleration component along that axis has a magnitude close to `1 g`, while the angular-velocity readings are near `0 dps`. The values may vary due to module orientation, zero-rate offset, and sensor noise [1].

Next, tilt the module to observe acceleration along each axis, then rotate it to observe changes in angular velocity. The readings need not match any fixed set of values; the purpose is to verify that the measurements respond appropriately to physical motion.

The following output was observed during hardware testing:

```
MPU6050 Sensor Monitor
----------------------
MPU6050 initialized.
Accel X: 0.060 g | Y: 0.005 g | Z: 1.030 g
Gyro X: 1.901 dps | Y: -1.534 dps | Z: -0.191 dps
Temperature: 46.412 C
----------------------
```

EXPECTED RESULT

The Serial Monitor displays acceleration and angular velocity along three axes, together with the internal MPU6050 temperature. When the module is tilted or rotated, the measurements change accordingly. When it is at rest, angular velocity is generally close to `0 dps`, and the accelerometer reflects the component of gravity along the sensor axes.

6 Extension

The main exercise uses the Beginner API with the default configuration. This section introduces selective-read and calibration functions, along with the Advanced API for selecting the I2C address, measurement ranges, and number of calibration samples [6].

6.1 Selective Reading and Status Checking

In addition to `MPU_ReadData()`, the Beginner API provides functions for reading individual groups of measurements and checking initialization status:

API	Function
`MPU_IsInitialized()`	Check the initialization state of the default device.
`MPU_ReadAcceleration()`	Read acceleration along the X, Y, and Z axes.
`MPU_ReadGyroscope()`	Read angular velocity about the X, Y, and Z axes.
`MPU_ReadTemperature()`	Read the internal MPU6050 sensor temperature.

Table 11.6.1. Selective-read and status-checking APIs

`MPU_ReadAcceleration()`, `MPU_ReadGyroscope()`, and `MPU_ReadTemperature()` read their respective measurement groups from the sensor. When all three groups are needed, `MPU_ReadData()` is more suitable because it updates them through a single API call [6].

Example when only acceleration is required:

```
float x = 0.0F;
float y = 0.0F;
float z = 0.0F;

if (true == MPU_ReadAcceleration(&x, &y, &z))
{
    /* Process acceleration data. */
}
```

6.2 Calibration with MPU_Calibrate()

`MPU_Calibrate()` calibrates the default device by collecting measurements while the sensor remains stationary and determining accelerometer and gyroscope zero-offset values. According to the EduFramework implementation, the Beginner API uses `500` samples and assumes a stable orientation with approximately `+1 g` along the `+Z` axis [6].

`MPU_Calibrate`

Calibrate the default MPU6050 and update the offsets applied to subsequent readings.

Syntax

```
MPU_Calibrate();
```

Return value

`true` if calibration completes successfully; `false` if the sensor has not been initialized or an error occurs during sample collection.

Note

Keep the module stationary throughout calibration and orient it as required by the algorithm. Moving the sensor while samples are being collected makes the calibration results unrepresentative of the intended static condition.

6.3 I2C Address and the AD0 Pin

When `AD0` is low, the MPU6050 uses the 7-bit I2C address `0x68`; when the pin is high, the address changes to `0x69`. Two MPU6050 devices can therefore be distinguished on the same I2C bus if their `AD0` pins are held at different logic levels [3].

EduFramework defines two address constants:

```
MPU6050_ADDRESS_DEFAULT   /* 0x68 */
MPU6050_ADDRESS_ALT       /* 0x69 */
```

The Beginner API uses the default address `0x68`. To select another address, the application must use the Advanced API with an `MPU6050_t` device context; the Wire layer must be initialized before calling `MPU6050_begin()` [6].

Example:

```
MPU6050_t mpu;
Wire_begin();
if (true == MPU6050_begin(&mpu, MPU6050_ADDRESS_ALT))
{
    /* Device at address 0x69 is ready. */
}
```

Listing 11.6.1. Initializing the MPU6050 with a custom I2C address

6.4 Customizing Measurement Ranges

The MPU6050 Product Specification defines four accelerometer full-scale ranges and four gyroscope full-scale ranges. EduFramework provides access to these settings through the Advanced API [3] [6].

Accelerometer	Gyroscope
`MPU6050_RANGE_2_G`	`MPU6050_RANGE_250_DEG`
`MPU6050_RANGE_4_G`	`MPU6050_RANGE_500_DEG`
`MPU6050_RANGE_8_G`	`MPU6050_RANGE_1000_DEG`
`MPU6050_RANGE_16_G`	`MPU6050_RANGE_2000_DEG`

Table 11.6.2. MPU6050 measurement-range options

The corresponding configuration APIs are:

```
MPU6050_setAccelerometerRange(&mpu, MPU6050_RANGE_4_G);
MPU6050_setGyroRange(&mpu, MPU6050_RANGE_500_DEG);
```

A larger full-scale range allows the sensor to represent greater motion amplitudes, but the sensitivity (counts per unit) also changes. The range should therefore be selected according to the expected acceleration or angular velocity rather than always choosing the largest setting [3].

6.5 Context-Based Advanced API

The Advanced API uses an `MPU6050_t` context to manage the I2C address, measurement ranges, sensor data, and calibration offsets instead of relying on the default configuration [6].

The initialization, configuration, and measurement APIs are summarized below:

Advanced API	Role
`MPU6050_begin()`	Initialize a context with the selected I2C address.
`MPU6050_setAccelerometerRange()`	Change the accelerometer range.
`MPU6050_setGyroRange()`	Change the gyroscope range.
`MPU6050_read()`	Read a measurement frame into the context.
`MPU6050_calibrate()`	Calibrate using a sample count selected by the application.

Table 11.6.3. Initialization, configuration, and measurement APIs

The APIs for retrieving measurements stored in the context are:

Advanced API	Role
`MPU6050_getAccelerationX()`, `MPU6050_getAccelerationY()`, `MPU6050_getAccelerationZ()`	Retrieve the stored acceleration component for each axis.
`MPU6050_getGyroX()`, `MPU6050_getGyroY()`, `MPU6050_getGyroZ()`	Retrieve the stored angular-velocity component for each axis.
`MPU6050_getTemperature()`	Retrieve the stored temperature.

Table 11.6.4. APIs for retrieving data from a device context

The Beginner API is suitable for one sensor operating with the default configuration. The Advanced API supports custom I2C addresses, measurement ranges, calibration sample counts, and separate device contexts.

6.6 Extension Exercise

Set `AD0` to a logic-high level to select address `0x69`, then use the Advanced API to:

- Initialize a separate `MPU6050_t` context;
- Select an accelerometer range of `±4 g`;
- Select a gyroscope range of `±500 dps`;
- Calibrate using a sample count selected by the program;
- Read and display acceleration and angular-velocity data on the Serial Monitor.

Do not call `MPU_Begin()` in this exercise. Use `MPU6050_begin()`, the range-configuration functions, and `MPU6050_calibrate()` through the Advanced API.

EXPECTED RESULT

The sensor initializes at address `0x69` and returns measurements after the ranges are configured. With calibration performed under the required stationary conditions, the zero-rate offset in the gyroscope readings may be reduced.

7 References

[1] University of California, Berkeley, *Strapdown inertial navigation* , Rotations.

[2] Cornell University, *Inter-Integrated Circuit (I2C)* , ECE 4760 - Designing with Microcontrollers.

[3] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification* , PS-MPU-6000A-00, Rev. 3.4, 2013.

[4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.

[5] FPT Software, *MaaZ Edu Development Board - User Manual* , Rev. 1.0, 2025.

[6] EduFramework, *MPU6050 Device API* , EduFramework Source Code.